

Unit 4. Resampling and Model Evaluation

Business Data Analytics: Introductory Machine Learning

Training error is not generalization error

Training error measures fit on data the model has seen; generalization error measures prediction on data it has not. The two diverge whenever a model can overfit. Cross-validation estimates the second; its assumptions fail silently.

1 The Generalization Question

This section fixes notation: the data-generating distribution, the iid sampling assumption, and the population vs. empirical risk distinction.

1.1 Population risk and empirical risk

OLS, ridge, and logistic regression are fit by minimizing an empirical risk on the training sample (Unit 1 §1.9.1; Unit 2 §2.1.3; Unit 3 §3.1.3). Deployment depends on a different quantity.

Definition 4.1 (Data-generating distribution and iid sampling). The pair $(X, Y) \in \mathbb{R}^p \times \mathcal{Y}$ has joint distribution P , the **data-generating distribution**. The training data $\{(X_i, Y_i)\}_{i=1}^n$ are an **iid sample** from P : the (X_i, Y_i) are independent across i , and each has distribution P .

Lowercase (x_i, y_i) denotes a realization of (X_i, Y_i) ; a row is written uppercase or lowercase depending on whether we reason about it before or after sampling.

Definition 4.2 (Population risk). Let $\ell : \mathcal{Y} \times \mathcal{Y} \rightarrow \mathbb{R}_{\geq 0}$ be a loss function (Unit 1 Definition 1.17) and $f : \mathbb{R}^p \rightarrow \mathcal{Y}$ a candidate predictor. The **population risk** of f under P is

$$\mathcal{R}(f) := \mathbb{E}_{(X,Y) \sim P} \ell(Y, f(X)),$$

the expected loss on a fresh draw from P . The **empirical risk** on the sample is

$$\widehat{\mathcal{R}}_n(f) := \frac{1}{n} \sum_{i=1}^n \ell(Y_i, f(X_i)),$$

recovering Unit 1 Definition 1.17 when f is the parameterized f_θ .

$\widehat{\mathcal{R}}_n(f)$ is a Monte Carlo estimate of $\mathcal{R}(f)$: the population expectation replaced by a sample average.

Lemma 4.3 (Empirical risk is unbiased on a fixed predictor). If f does not depend on the sample $\{(X_i, Y_i)\}_{i=1}^n$, then

$$\mathbb{E} \hat{\mathcal{R}}_n(f) = \mathcal{R}(f).$$

Proof. By linearity of expectation and identical distribution,

$$\mathbb{E} \hat{\mathcal{R}}_n(f) = \frac{1}{n} \sum_{i=1}^n \mathbb{E} \ell(Y_i, f(X_i)) = \mathbb{E} \ell(Y, f(X)) = \mathcal{R}(f).$$

The independence of the (X_i, Y_i) across i is not used; only their identical distribution is needed. $\square$

Where the proof breaks for fitted models

When $f = f_{\hat{\theta}}$ is fitted by minimizing $\hat{\mathcal{R}}_n$ on the same sample, the predictor depends on the sample: $\hat{\theta}$ was chosen to make $\hat{\mathcal{R}}_n(f_{\hat{\theta}})$ small. The lemma's hypothesis (f does not depend on the sample) fails, and $\hat{\mathcal{R}}_n(f_{\hat{\theta}})$ systematically underestimates $\mathcal{R}(f_{\hat{\theta}})$.

1.2 The optimism of training error

Definition 4.4 (Training error and optimism). The **training error** of a fitted predictor $f_{\hat{\theta}}$ is $\hat{\mathcal{R}}_n(f_{\hat{\theta}})$, the empirical risk on the data used to fit it. The **optimism** is

$$\text{op}(\hat{\theta}) := \mathcal{R}(f_{\hat{\theta}}) - \hat{\mathcal{R}}_n(f_{\hat{\theta}}).$$

Lemma 4.3 gives $\mathbf{E} \text{ op} = 0$ for any f that does not depend on the sample. When $\hat{\theta}$ minimizes $\hat{\mathcal{R}}_n$, it absorbs sample noise, so $\hat{\mathcal{R}}_n(f_{\hat{\theta}}) < \mathcal{R}(f_{\hat{\theta}})$ on average. Expected optimism is positive and grows with model flexibility.

Example 4.1 (OLS on pure noise). Fit OLS on $n = 50$ rows of $p = 20$ random predictors against a target with no signal; compare training mean squared error (MSE) to test MSE on a fresh sample of $N = 10^4$.

```
1 set.seed(2026)
2 n <- 50; p <- 20; N <- 1e4
3 sim_one <- function() {
4   X_tr <- matrix(rnorm(n * p), n, p); y_tr <- rnorm(n)
5   X_te <- matrix(rnorm(N * p), N, p); y_te <- rnorm(N)
```

```

6  beta <- lm.fit(X_tr, y_tr)$coefficients
7  c(train = mean((y_tr - X_tr %*% beta)^2),
8    test  = mean((y_te - X_te %*% beta)^2))
9  }
10 colMeans(t(replicate(200, sim_one()))))

```

```

##      train      test
## 0.604072 1.679585

```

The signal is zero: any predictor that ignores X has $\mathcal{R} = 1$. Training MSE averages well below 1 (the fit absorbs sample noise); test MSE averages well above 1 (that noise carries no predictive value out of sample). The gap is the optimism.

Theorem 4.5 (In-sample optimism for least squares). Fix design X ; let $y \in \mathbb{R}^n$ have $\mathbf{E} y = \mu$ and $\text{cov } y = \sigma^2 I_n$, and let $y' \in \mathbb{R}^n$ be an independent copy of y . For any predictor $\hat{y} = g(y)$ with finite second moments,

$$\frac{1}{n} \mathbf{E} \|y' - \hat{y}\|^2 - \frac{1}{n} \mathbf{E} \|y - \hat{y}\|^2 = \frac{2}{n} \sum_{i=1}^n \text{cov}(y_i, \hat{y}_i).$$

Under Unit 2 Assumption A2 (full column rank of X), OLS gives $\hat{y} = P_X y$ with $P_X = X(X^\top X)^{-1}X^\top$, and the right-hand side equals $2\sigma^2 p/n$: one σ^2 per fitted parameter, divided by sample size.

Proof. General identity. Expand the squares and use $\mathbb{E}(y'_i)^2 = \mathbb{E} y_i^2$, $\mathbb{E} y'_i = \mathbb{E} y_i$, and $y'_i \perp \hat{y}_i$:

$$\begin{aligned} \mathbb{E}(y'_i - \hat{y}_i)^2 - \mathbb{E}(y_i - \hat{y}_i)^2 &= -2(\mathbb{E}[y'_i \hat{y}_i] - \mathbb{E}[y_i \hat{y}_i]) \\ &= 2(\mathbb{E}[y_i \hat{y}_i] - \mathbb{E} y_i \cdot \mathbb{E} \hat{y}_i) \\ &= 2 \operatorname{cov}(y_i, \hat{y}_i). \end{aligned}$$

Sum over i and divide by n ; since $\sum_i \mathbb{E}(y_i - \hat{y}_i)^2 = \mathbb{E} \|y - \hat{y}\|^2$ and likewise for y' ,

$$\frac{1}{n} \mathbb{E} \|y' - \hat{y}\|^2 - \frac{1}{n} \mathbb{E} \|y - \hat{y}\|^2 = \frac{2}{n} \sum_{i=1}^n \operatorname{cov}(y_i, \hat{y}_i).$$

OLS specialization. P_X is deterministic in y , and $\operatorname{cov} y = \sigma^2 I$ gives

$$\operatorname{cov}(y_i, \hat{y}_i) = \operatorname{cov}\left(y_i, \sum_j (P_X)_{ij} y_j\right) = \sum_j (P_X)_{ij} \operatorname{cov}(y_i, y_j) = \sigma^2 (P_X)_{ii}.$$

Sum and apply the cyclic trace property:

$$\sum_{i=1}^n \text{cov}(y_i, \hat{y}_i) = \sigma^2 \text{tr}(P_X) = \sigma^2 \text{tr}((X^\top X)^{-1} X^\top X) = \sigma^2 \text{tr}(I_p) = \sigma^2 p.$$

□

When $\hat{y}$ absorbs sample noise in y , each $\hat{y}_i$ acquires positive correlation with y_i ; the optimism is twice the row-average of these covariances. The same identity defines the **effective degrees of freedom** of Unit 2 §2.2.3 and §2.5.2 and underlies Mallows's C_p and AIC. The simulation above runs in the random- X regime, where the gap is typically larger.

1.3 The single train/test split and its limits

A held-out test set satisfies the hypothesis of Lemma 4.3. Partition the data into disjoint training and test indices T and V (Unit 1 Definition 1.2), fit $\hat{\theta}$ on $\{(X_i, Y_i) : i \in T\}$, and report

$$\widehat{\mathcal{R}}_V(f_{\hat{\theta}}) := \frac{1}{|V|} \sum_{i \in V} \ell(Y_i, f_{\hat{\theta}}(X_i)).$$

Since $T \cap V = \emptyset$, $f_{\hat{\theta}}$ (a function of training data only) does not depend on the test sample. Applying Lemma 4.3 conditional on the training data, $\widehat{\mathcal{R}}_V(f_{\hat{\theta}})$ is an unbiased estimator of $\mathcal{R}(f_{\hat{\theta}})$.

Unbiasedness alone is not enough. $\widehat{\mathcal{R}}_V(f_{\hat{\theta}})$ is a sample average of $|V|$ loss values, with standard error of order $\sigma_{\ell}/\sqrt{|V|}$, where $\sigma_{\ell}^2 = \text{var } \ell(Y, f_{\hat{\theta}}(X))$. The example below shows the spread for a 70/30 split.

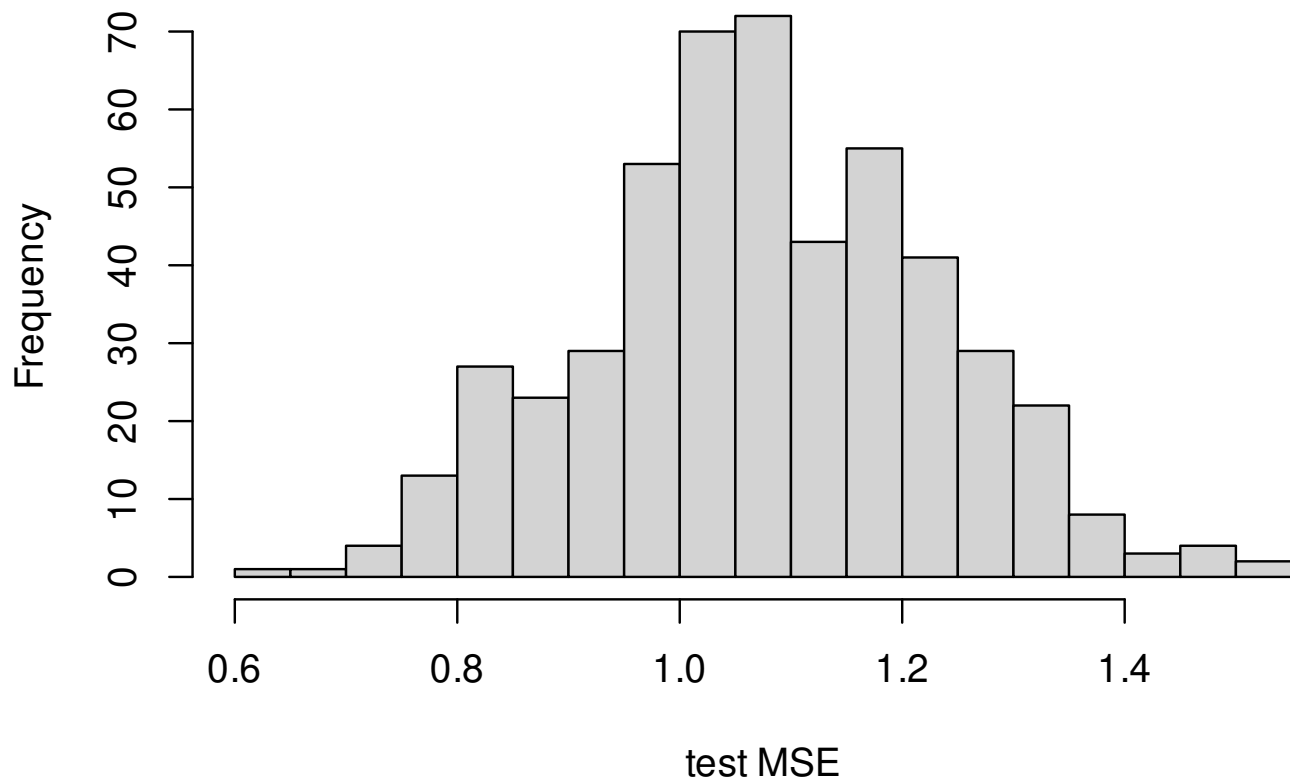
Example 4.2 (Variability of test MSE across splits). Fix a sample of size $n = 200$ with a linear signal; do 70/30 splits 500 times, refit each, and record the test MSE.

```
1 set.seed(1)
2 n <- 200; p <- 5
3 beta <- c(1, -1, 0.5, 0, 0)
4 X <- matrix(rnorm(n * p), n, p)
5 y <- X %*% beta + rnorm(n)
6 mse_split <- function() {
7   V <- sample(n, 60); T <- setdiff(seq_len(n), V)
8   fit <- lm.fit(X[T, ], y[T])
9   mean((y[V] - X[V, ] %*% fit$coefficients)^2)
10 }
```

```
11 test_mse <- replicate(500, mse_split())
12 c(mean = mean(test_mse), sd = sd(test_mse))
```

```
##          mean          sd
## 1.0758807 0.1558926
```

```
1 par(mar = c(4, 5, 0.5, 0.5))
2 hist(test_mse, breaks = 30, main = NULL, xlab = "test MSE")
```



The 500 values cluster around the population risk with substantial spread; one 70/30 split often

yields a test MSE far from $\mathcal{R}(f_{\hat{\theta}})$. Reporting it as the test error ignores this variability.

The case for resampling

Each split gives an unbiased but noisy estimate. Averaging across multiple splits reduces variance without introducing bias. That is the role of cross-validation.

2 Bias–Variance Decomposition

2.1 The decomposition for squared loss

Throughout this section, $(X, Y) \sim P$, $\mathcal{D} = \{(X_i, Y_i)\}_{i=1}^n$ is iid from P , and $\hat{f} = \hat{f}_{\mathcal{D}}$ is the predictor fitted on $\mathcal{D}$. The test pair (X, Y) is independent of $\mathcal{D}$. Subscripts on $\mathbf{E}$ indicate the source averaged: $\mathbf{E}_{\mathcal{D}}$ over $\mathcal{D}$, $\mathbf{E}_{Y|x}$ over $Y \mid X = x$; unsubscripted $\mathbf{E}[\cdot \mid X = x]$ averages over both.

Definition 4.6 (Regression function and conditional noise variance). The **regression function** of P is

$$f^*(x) := \mathbf{E}[Y \mid X = x].$$

The **conditional noise variance** at x is $\sigma^2(x) := \text{var}(Y \mid X = x)$. By definition, $\mathbf{E}[Y - f^*(x) \mid X = x] = 0$ and $\text{var}(Y - f^*(x) \mid X = x) = \sigma^2(x)$.

Definition 4.7 (Pointwise bias and variance of $\hat{f}$). For a fitted predictor $\hat{f}$ and a fixed test point x ,

$$\text{bias}(\hat{f}, x) := \mathbf{E}_{\mathcal{D}}[\hat{f}(x)] - f^*(x), \quad \text{var}(\hat{f}, x) := \text{var}_{\mathcal{D}}(\hat{f}(x)).$$

Theorem 4.8 (Pointwise bias-variance decomposition). For a fixed test point x and $\hat{f}$ fitted on $\mathcal{D}$ drawn independently of the test pair (X, Y) ,

$$\mathbf{E}[(Y - \hat{f}(x))^2 \mid X = x] = \text{bias}(\hat{f}, x)^2 + \text{var}(\hat{f}, x) + \sigma^2(x).$$

Proof. Separate the noise. Write $Y - \hat{f}(x) = (Y - f^*(x)) + (f^*(x) - \hat{f}(x))$, square, and

take $\mathbf{E}[\cdot \mid X = x]$:

$$\begin{aligned}\mathbf{E}[(Y - \hat{f}(x))^2 \mid X = x] &= \mathbf{E}[(Y - f^*(x))^2 \mid X = x] \\ &\quad + 2 \mathbf{E}[(Y - f^*(x))(f^*(x) - \hat{f}(x)) \mid X = x] \\ &\quad + \mathbf{E}[(f^*(x) - \hat{f}(x))^2 \mid X = x].\end{aligned}$$

The first term is $\sigma^2(x)$ by Definition 4.6. The cross term vanishes: $\mathcal{D}$ is independent of $Y \mid X = x$, so the expectation factors, and $\mathbf{E}_{Y|x}[Y - f^*(x)] = 0$. The third term reduces to $\mathbf{E}_{\mathcal{D}}[(f^*(x) - \hat{f}(x))^2]$ since $\hat{f}(x)$ does not depend on Y .

Decompose the model term. Insert $\pm \mathbf{E}_{\mathcal{D}} \hat{f}(x)$:

$$f^*(x) - \hat{f}(x) = \underbrace{(f^*(x) - \mathbf{E}_{\mathcal{D}} \hat{f}(x))}_{=-\text{bias}(\hat{f}, x), \text{ constant in } \mathcal{D}} + \underbrace{(\mathbf{E}_{\mathcal{D}} \hat{f}(x) - \hat{f}(x))}_{\mathbf{E}_{\mathcal{D}}=0}.$$

Square and take $\mathbf{E}_{\mathcal{D}}[\cdot]$; the cross term vanishes (constant times mean-zero), leaving

$$\mathbf{E}_{\mathcal{D}}[(f^*(x) - \hat{f}(x))^2] = \text{bias}(\hat{f}, x)^2 + \text{var}(\hat{f}, x).$$

Combining the two steps yields the claim. □

2.2 Integrated risk and the trade-off

Apply $\mathbf{E}_X[\cdot]$ to both sides of Theorem 4.8. The pointwise LHS already averages over $Y \mid x$ and $\mathcal{D}$, so applying $\mathbf{E}_X$ gives the expectation over all three sources. Conditioning on $\mathcal{D}$ and using $\mathcal{D} \perp (X, Y)$:

$$\begin{aligned} \mathbf{E}_X \left[\mathbf{E}[(Y - \hat{f}(X))^2 \mid X] \right] &= \mathbf{E}_{X,Y,\mathcal{D}}[(Y - \hat{f}(X))^2] \\ &= \mathbf{E}_{\mathcal{D}} \left[\mathbf{E}_{(X,Y) \sim P}[(Y - \hat{f}_{\mathcal{D}}(X))^2] \right] \\ &= \mathbf{E}_{\mathcal{D}}[\mathcal{R}(\hat{f})]. \end{aligned}$$

The inner expectation in the second line treats $\hat{f}_{\mathcal{D}}$ as fixed (we conditioned on $\mathcal{D}$) and is the population risk of that fixed function (Definition 4.2). The outer $\mathbf{E}_{\mathcal{D}}$ appears because $\mathcal{R}(\hat{f})$ is itself random through $\hat{f} = \hat{f}_{\mathcal{D}}$: the decomposition is for the population risk averaged across training samples, not the realized risk on one $\mathcal{D}$.

Applying $\mathbf{E}_X$ term by term to the RHS of Theorem 4.8 gives the matching identity

$$\mathbf{E}_{\mathcal{D}}[\mathcal{R}(\hat{f})] = \mathbf{E}_X[\text{bias}(\hat{f}, X)^2] + \mathbf{E}_X[\text{var}(\hat{f}, X)] + \mathbf{E}_X[\sigma^2(X)].$$

The three terms are average squared bias, average variance, and irreducible noise. The first two depend

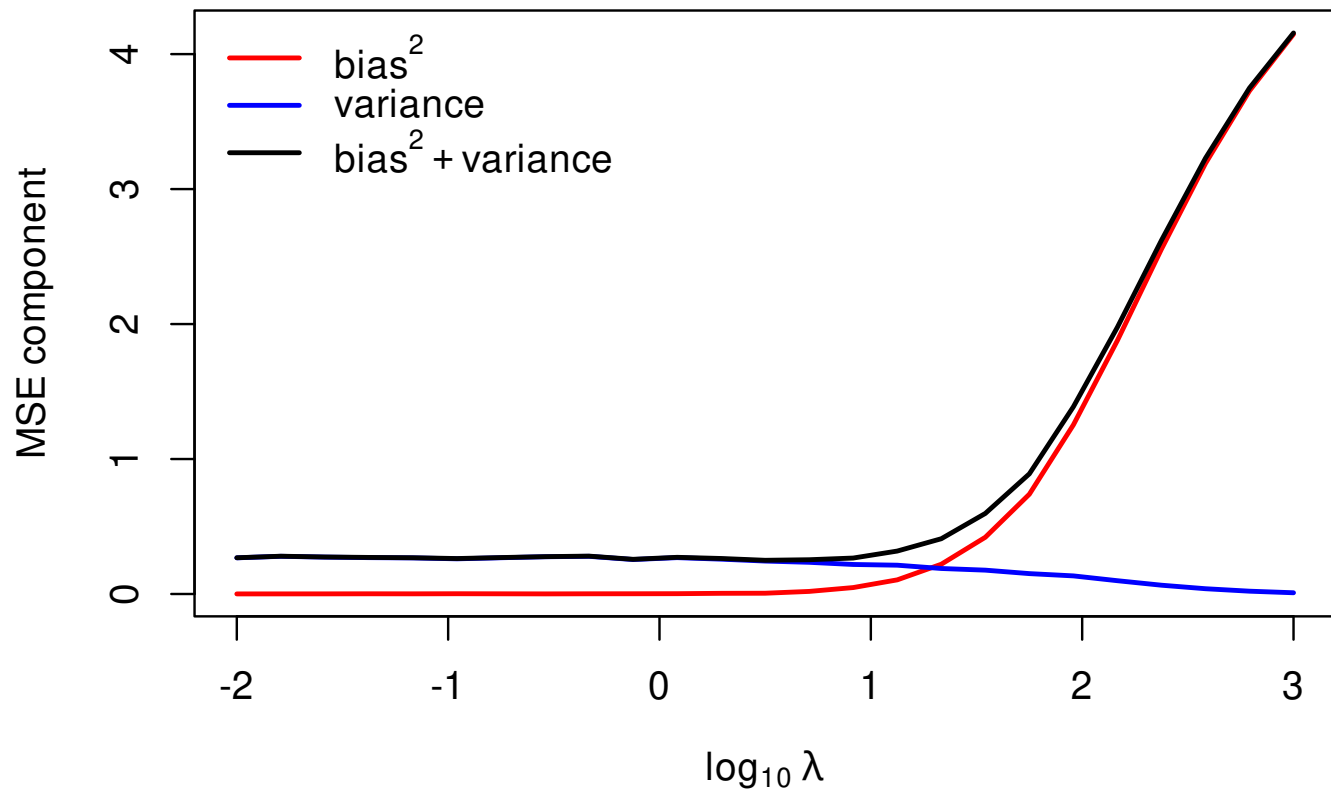
on $\hat{f}$: for ridge regression (Unit 2 §2.2), increasing λ shrinks coefficients, raising bias and lowering variance. Their sum is U-shaped in λ ; the minimum is the optimal λ .

Example 4.3 (Bias-variance trade-off for ridge). Truth: $y = X\beta^* + \varepsilon$ with $p = 20$, $n = 100$, $\sigma = 1$, $\beta^* = (2, -1, 0.5, 0, \dots, 0)$. Across a grid of λ , estimate $\mathbf{E}_X \text{bias}^2$ and $\mathbf{E}_X \text{var}$ by averaging over $B = 200$ training samples at $n_{\text{te}} = 200$ fixed test points.

```
1 set.seed(1)
2 n <- 100; p <- 20; B <- 200; n_te <- 200
3 beta_star <- c(2, -1, 0.5, rep(0, p - 3))
4 X_test <- matrix(rnorm(n_te * p), n_te, p)
5 y_test <- as.numeric(X_test %*% beta_star)
6 lambdas <- 10^seq(-2, 3, length.out = 25)
7
8 ridge_fit <- function(X, y, lambda)
9   solve(crossprod(X) + lambda * diag(p), crossprod(X, y))
10
11 bv_one <- function(lambda) {
12   preds <- replicate(B, {
13     X_tr <- matrix(rnorm(n * p), n, p)
```



```
14   y_tr <- as.numeric(X_tr %*% beta_star + rnorm(n))
15   as.numeric(X_test %*% ridge_fit(X_tr, y_tr, lambda))
16 })
17 c(bias2      = mean((rowMeans(preds) - y_test)^2),
18    variance = mean(apply(preds, 1, var)))
19 }
20 res <- t(sapply(lambdas, bv_one))
21
22 par(mar = c(4, 5, 0.5, 0.5))
23 matplot(log10(lambdas), cbind(res, rowSums(res)), type = "l", lty = 1,
24        col = c("red", "blue", "black"), lwd = 2,
25        xlab = expression(log[10]~lambda), ylab = "MSE component")
26 legend("topleft", legend = c(expression(bias^2), "variance",
27                               expression(bias^2 + variance)),
28        col = c("red", "blue", "black"), lty = 1, lwd = 2, bty = "n")
```



As λ grows, bias rises and variance falls; their sum has a U-shape with an interior minimum.

Cross-validation estimates this curve from a single sample and selects the minimizing λ (Unit 2 §2.5.1).

2.3 0/1 loss and the limits of the decomposition

Why the decomposition does not transfer to classification

Theorem 4.8 relies on the quadratic identity $(a + b)^2 = a^2 + 2ab + b^2$; no analogous identity exists for 0/1 loss. Proposed bias-variance decompositions for classification (Domingos 2000; Kohavi and Wolpert 1996; Tibshirani 1996) each fail at least one of: (i) all terms nonnegative; (ii) bias zero corresponds to the population-optimal classifier; (iii) reduction to Theorem 4.8 for squared loss. Classification model selection in this course relies on Unit 3's metrics and on resampling, not on a decomposition.

3 K-fold Cross-Validation

3.1 The K-fold CV estimator

A single train/test split (§1.3) gave an unbiased estimator with high variance. K-fold cross-validation repeats the split in a structured way and averages.

Definition 4.9 (K-fold partition). A **K-fold partition** of $\{1, \dots, n\}$ is a sequence of disjoint subsets $\mathcal{I}_1, \dots, \mathcal{I}_K$ with $\bigcup_{k=1}^K \mathcal{I}_k = \{1, \dots, n\}$. When formed by uniform random shuffling, the fold sizes differ from n/K by at most one.

For fold k , $\hat{f}^{(-k)}$ denotes the predictor fitted on the rows not in $\mathcal{I}_k$ (training size $n - |\mathcal{I}_k|$). The model is evaluated on $\mathcal{I}_k$; cycling k and averaging gives the CV estimator.

Definition 4.10 (K-fold CV estimator). The **K-fold cross-validation estimator** is

$$\text{CV}_K := \frac{1}{K} \sum_{k=1}^K \widehat{\mathcal{R}}_{\mathcal{I}_k}(\hat{f}^{(-k)}) = \frac{1}{K} \sum_{k=1}^K \frac{1}{|\mathcal{I}_k|} \sum_{i \in \mathcal{I}_k} \ell(Y_i, \hat{f}^{(-k)}(X_i)).$$

When the folds are equal-sized, this simplifies to

$$\text{CV}_K = \frac{1}{n} \sum_{i=1}^n \ell(Y_i, \hat{f}^{(-k(i))}(X_i)),$$

where $k(i)$ is the fold containing observation i ; each row contributes once, evaluated by the model that did not see it.

Iter 1:	Test	Train	Train	Train	Train
Iter 2:	Train	Test	Train	Train	Train
Iter 3:	Train	Train	Test	Train	Train
Iter 4:	Train	Train	Train	Test	Train
Iter 5:	Train	Train	Train	Train	Test

Figure 4.1: $K = 5$ -fold cross-validation: each iteration tests on one fold (orange) and trains on the rest (blue).

Each per-fold loss $\ell(Y_i, \hat{f}^{(-k)}(X_i))$ for $i \in \mathcal{I}_k$ is computed on a row independent of $\hat{f}^{(-k)}$. Conditioning on $\hat{f}^{(-k)}$, Lemma 4.3 gives

$$\mathbb{E}[\widehat{\mathcal{R}}_{\mathcal{I}_k}(\hat{f}^{(-k)}) \mid \hat{f}^{(-k)}] = \mathcal{R}(\hat{f}^{(-k)}).$$

Taking total expectation: $\mathbb{E}[\widehat{\mathcal{R}}_{\mathcal{I}_k}(\hat{f}^{(-k)})] = \mathbb{E}[\mathcal{R}(\hat{f}^{(-k)})]$, with the latter the same for every k by symmetry. Averaging,

$$\mathbb{E}[\text{CV}_K] = \mathbb{E}[\mathcal{R}(\hat{f}^{(-k)})] \quad (\text{any } k),$$

the expected risk of a model fitted on $n - |\mathcal{I}_k| \approx n(K - 1)/K$ rows. The gap from $\mathbb{E}_{\mathcal{D}}[\mathcal{R}(\hat{f}_{\mathcal{D}})]$ at training size n is the bias of K-fold CV.

3.2 Bias of K-fold CV

CV_K estimates the expected risk at training size $n(K-1)/K$, but the deployed model is fit on all n training rows. For procedures whose expected risk decreases with training size, $\mathbf{E}[CV_K] > \mathbf{E}[\mathcal{R}(\hat{f}_n)]$: the bias is positive, and CV is **pessimistic**.

The size of the bias decreases in K . For OLS the formula is exact:

Bias of K-fold CV for OLS at fixed design

Expected risk at training size m . For OLS with $\text{cov } y = \sigma^2 I_m$ and correctly specified mean $\mu := \mathbf{E} y \in \text{col}(X)$, the fitted vector $\hat{y} = P_X y$ satisfies $\hat{y} - \mu = P_X(y - \mu)$. Risk on a fresh in-sample y' :

$$\mathbf{E}[\|y' - \hat{y}\|^2/m] = \underbrace{\mathbf{E} \|y' - \mu\|^2/m}_{=\sigma^2} + \underbrace{\mathbf{E} \|\mu - \hat{y}\|^2/m}_{=\sigma^2 p/m},$$

the cross term vanishing because $y' \perp \hat{y}$ with $\mathbf{E}(y' - \mu) = 0$; the second term by $\mathbf{E} \|P_X(y - \mu)\|^2 = \sigma^2 \text{tr}(P_X) = \sigma^2 p$. Thus $\mathbf{E}[\mathcal{R}(\hat{f}_m)] = \sigma^2(1 + p/m)$.

Bias of CV. Substituting $m = n(K - 1)/K$ for the per-fold training size,

$$\text{bias}(\text{CV}_K) = \mathbb{E}[\mathcal{R}(\hat{f}_m)] - \mathbb{E}[\mathcal{R}(\hat{f}_n)] = \sigma^2 p \left(\frac{1}{m} - \frac{1}{n} \right) = \frac{\sigma^2 p}{n(K - 1)}.$$

At $\sigma^2 = 1$, $p = 20$, $n = 200$:

K	2	5	10	20	n (LOO)
$\text{bias}(\text{CV}_K)$	0.10	0.025	0.011	0.0053	≈ 0

The bias falls as $1/(K - 1)$: $K = 2$ overestimates the risk of the deployed model by about 9%, $K = 10$ by about 1%, and LOO eliminates it.

3.3 Variance of K-fold CV and the LOO trade

Bias decreases monotonically in K ; variance does not. As K grows, the per-fold training sets $[n] \setminus \mathcal{I}_k$ become more similar (sharing a fraction $(K - 2)/K$ of their rows), so the per-fold losses $L_k := \hat{\mathcal{R}}_{\mathcal{I}_k}(\hat{f}^{(-k)})$ become correlated. Averaging correlated quantities reduces variance less than averaging

independent ones.

Variance with correlated folds

By exchangeability, $\text{var}(L_k) = \sigma_L^2$ and $\text{cov}(L_j, L_k) = \rho_K \sigma_L^2$ for $j \neq k$. Then

$$\text{var}(\text{CV}_K) = \frac{1}{K^2} (K\sigma_L^2 + K(K-1)\rho_K\sigma_L^2) = \frac{\sigma_L^2}{K} (1 + (K-1)\rho_K).$$

For $\rho_K = 0$, $\text{var} = \sigma_L^2/K$ (standard $1/K$ averaging). As $\rho_K \rightarrow 1$, $\text{var} \rightarrow \sigma_L^2$, no better than a single fold.

LOO is the extreme: training sets share $(n-2)/(n-1) \approx 1$ of their rows, so $\hat{f}^{(-j)}$ and $\hat{f}^{(-k)}$ are **nearly identical functions of the data**, forcing $\rho_K \approx 1$. Whether this makes $\text{var}(\text{CV}_{\text{LOO}})$ large depends on the model class:

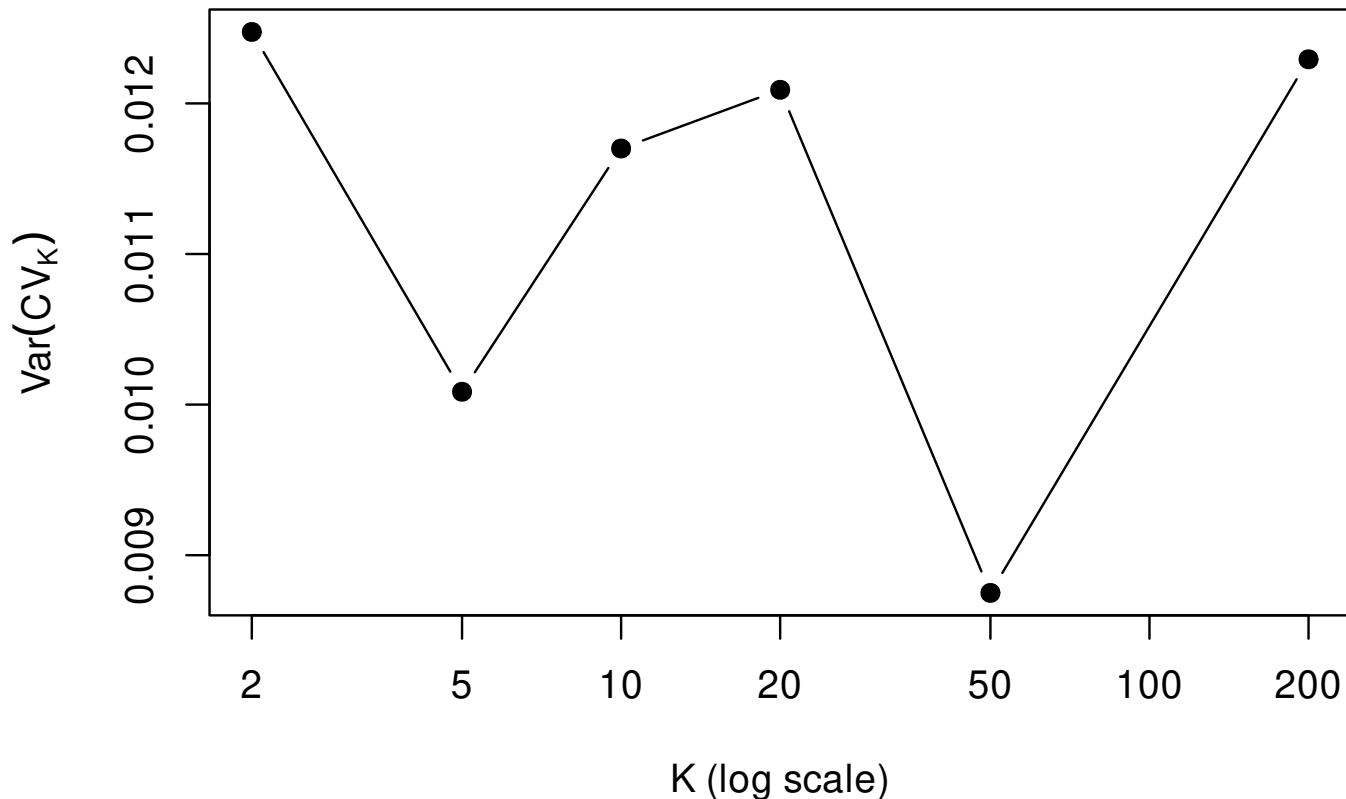
Model class	LOO behavior
Stable (OLS at $n \gg p$, ridge)	σ_L^2 small per row; LOO variance comparable to moderate- K
Unstable (nearest neighbors, single trees)	σ_L^2 large; LOO inflated relative to $K = 10$

Standard choice: $K = 5$ or $K = 10$. Bias is below $\sim 1\%$ at $K = 10$, and the cost is K full fits, not n . LOO is reserved for closed-form shortcuts (the OLS leverage formula) or settings where σ_L^2 is small enough to make the correlated average reliable.

Example 4.4 (Variance of CV_K across data samples for OLS). $n = 200$, $p = 5$, linear signal as in §1.3; for each K , CV_K is computed on $B = 100$ independent datasets and the across-replicate variance recorded.

```
1 set.seed(1)
2 n <- 200; p <- 5; B <- 100
3 beta <- c(1, -1, 0.5, 0, 0)
4 Ks <- c(2, 5, 10, 20, 50, 200)
5
6 cv_K <- function(K, X, y) {
7   fold_id <- sample(rep(1:K, length.out = n))
8   errs <- vapply(1:K, function(k) {
9     tr <- which(fold_id != k); te <- which(fold_id == k)
10    fit <- lm.fit(X[tr, , drop = FALSE], y[tr])
11    mean((y[te] - X[te, , drop = FALSE] %*% fit$coefficients)^2)
12  }, numeric(1))
```

```
13   mean(errs)
14 }
15
16 result <- sapply(Ks, function(K) {
17   replicate(B, {
18     X <- matrix(rnorm(n * p), n, p)
19     y <- as.numeric(X %*% beta + rnorm(n))
20     cv_K(K, X, y)
21   })
22 })
23
24 par(mar = c(4, 5, 0.5, 0.5))
25 plot(Ks, apply(result, 2, var), type = "b", log = "x", pch = 19,
26       xlab = "K (log scale)", ylab = expression(Var(CV[K])))
```



For OLS at $n \gg p$, variance is roughly constant in K over this range (0.009–0.012): the σ_L^2/K

averaging effect approximately cancels the growth in fold correlation $(K-1)\rho_K$. $K=5$ or $K=10$ achieves small bias at variance comparable to LOO with much less computation.

$\text{var}(\text{CV}_K)$ cannot be estimated from the per-row CV losses $\{e_i\}$ alone:

Theorem 4.11 (Bengio and Grandvalet, 2004). Consider K-fold CV ($K \geq 2$) with equal folds of size $m = n/K$, and let $e_i := \ell(Y_i, \hat{f}^{(-k(i))}(X_i))$ be the per-row CV loss, where $k(i)$ is the fold containing i . Suppose the joint distribution of $(e_1, \dots, e_n)$ is invariant under within-fold permutations and under permutations of the fold labels (the natural symmetry inherited from iid sampling and equal-size partitioning). Then no symmetric quadratic statistic $T = T(e_1, \dots, e_n)$ satisfies $\mathbf{E} T = \text{var}(\text{CV}_K)$ for every joint distribution in this exchangeable class.

Proof. Variance decomposition. Exchangeability reduces the joint second-moment structure to three parameters:

$$\sigma^2 := \text{var}(e_i), \quad \gamma_w := \text{cov}(e_i, e_j) \text{ for } i \neq j, k(i) = k(j), \quad \gamma_b := \text{cov}(e_i, e_j) \text{ for } k(i) \neq k(j).$$

With K folds of size $m = n/K$, there are $Km(m-1) = n^2/K - n$ ordered same-fold pairs and

$n^2(1 - 1/K)$ ordered between-fold pairs. Therefore

$$\text{var}(\text{CV}_K) = \frac{1}{n^2} [n\sigma^2 + (n^2/K - n)\gamma_w + n^2(1 - 1/K)\gamma_b] = \frac{\sigma^2}{n} + \frac{n-K}{nK} \gamma_w + \frac{K-1}{K} \gamma_b.$$

General symmetric quadratic estimator. Imposing the same exchangeability on T , the most general quadratic form is

$$T = a_1 \sum_{i=1}^n e_i^2 + a_2 \sum_{\substack{i \neq j \\ k(i)=k(j)}} e_i e_j + a_3 \sum_{k(i) \neq k(j)} e_i e_j + a_4 \sum_{i=1}^n e_i + a_5.$$

Moment matching. Using $\mathbf{E} e_i^2 = \sigma^2 + \mu^2$, $\mathbf{E}[e_i e_j] = \gamma_w + \mu^2$ (same fold), $\mathbf{E}[e_i e_j] = \gamma_b + \mu^2$ (between folds), and $\mathbf{E} e_i = \mu$:

$$\mathbf{E} T = a_1 n(\sigma^2 + \mu^2) + a_2 (n^2/K - n)(\gamma_w + \mu^2) + a_3 n^2(1 - 1/K)(\gamma_b + \mu^2) + a_4 n\mu + a_5.$$

For $\mathbf{E} T = \text{var}(\text{CV}_K)$ to hold for all $(\mu, \sigma^2, \gamma_w, \gamma_b)$, match coefficients of each:

- Coefficient of σ^2 : $a_1 n = 1/n \Rightarrow a_1 = 1/n^2$.

- Coefficient of γ_w : $a_2(n^2/K - n) = (n - K)/(nK) \Rightarrow a_2 = 1/n^2$.
- Coefficient of γ_b : $a_3n^2(1 - 1/K) = (K - 1)/K \Rightarrow a_3 = 1/n^2$.
- Coefficients of μ and constant: RHS has neither μ nor a constant, so $a_4 = 0$ and $a_5 = 0$.
- Coefficient of μ^2 : $a_1n + a_2(n^2/K - n) + a_3n^2(1 - 1/K) = 0$ (RHS has no μ^2).

Contradiction. Substituting $a_1 = a_2 = a_3 = 1/n^2$ into the μ^2 constraint:

$$\frac{1}{n^2} [n + (n^2/K - n) + n^2(1 - 1/K)] = \frac{1}{n^2} \cdot n^2 = 1 \neq 0.$$

The system is inconsistent, so no symmetric quadratic estimator is unbiased. □

Reported CV standard errors are heuristic

Standard implementations report the across-fold SD divided by $\sqrt{K}$. This estimates within-run variability, not $\text{var}(\text{CV}_K)$; it tacitly assumes $\rho_K = 0$. Useful as a rough indicator, not for formal inference. The unbiased measurement is to repeat CV across independent data samples, as in Example 4.4.

3.4 Repeated K-fold

CV_K depends on which K-fold partition was drawn; averaging over R independent partitions removes this partition-level variance.

Definition 4.12 (R-repeated K-fold CV). Given R independent K-fold partitions of the same sample, with $CV_K^{(r)}$ the K-fold estimate from the r -th partition, the **R-repeated K-fold CV estimator** is

$$\overline{CV}_K^{(R)} := \frac{1}{R} \sum_{r=1}^R CV_K^{(r)}.$$

Lemma 4.13 (Variance of repeated K-fold). Let $V_{\text{data}} := \text{var}(\mathbf{E}[CV_K \mid \mathcal{D}])$ (data-level variance, averaging out the partition) and $V_{\text{part}} := \mathbf{E}[\text{var}(CV_K \mid \mathcal{D})]$ (partition-level variance, averaging out the data). Then

$$\text{var}(\overline{CV}_K^{(R)}) = V_{\text{data}} + \frac{V_{\text{part}}}{R}.$$

Proof. By the law of total variance, conditioning on $\mathcal{D}$:

$$\text{var}(\overline{\text{CV}}_K^{(R)}) = \text{var}(\mathbb{E}[\overline{\text{CV}}_K^{(R)} \mid \mathcal{D}]) + \mathbb{E}[\text{var}(\overline{\text{CV}}_K^{(R)} \mid \mathcal{D})].$$

Conditional on $\mathcal{D}$, the R partitions are iid, so $\mathbb{E}[\overline{\text{CV}}_K^{(R)} \mid \mathcal{D}] = \mathbb{E}[\text{CV}_K \mid \mathcal{D}]$ and $\text{var}(\overline{\text{CV}}_K^{(R)} \mid \mathcal{D}) = \text{var}(\text{CV}_K \mid \mathcal{D})/R$. Substituting yields the claim. $\square$

As $R \rightarrow \infty$, $\text{var}(\overline{\text{CV}}_K^{(R)}) \rightarrow V_{\text{data}}$: repetition cannot reduce the data-level component. The relative gain is governed by $V_{\text{part}}/V_{\text{data}}$, small for stable procedures and larger for unstable ones.

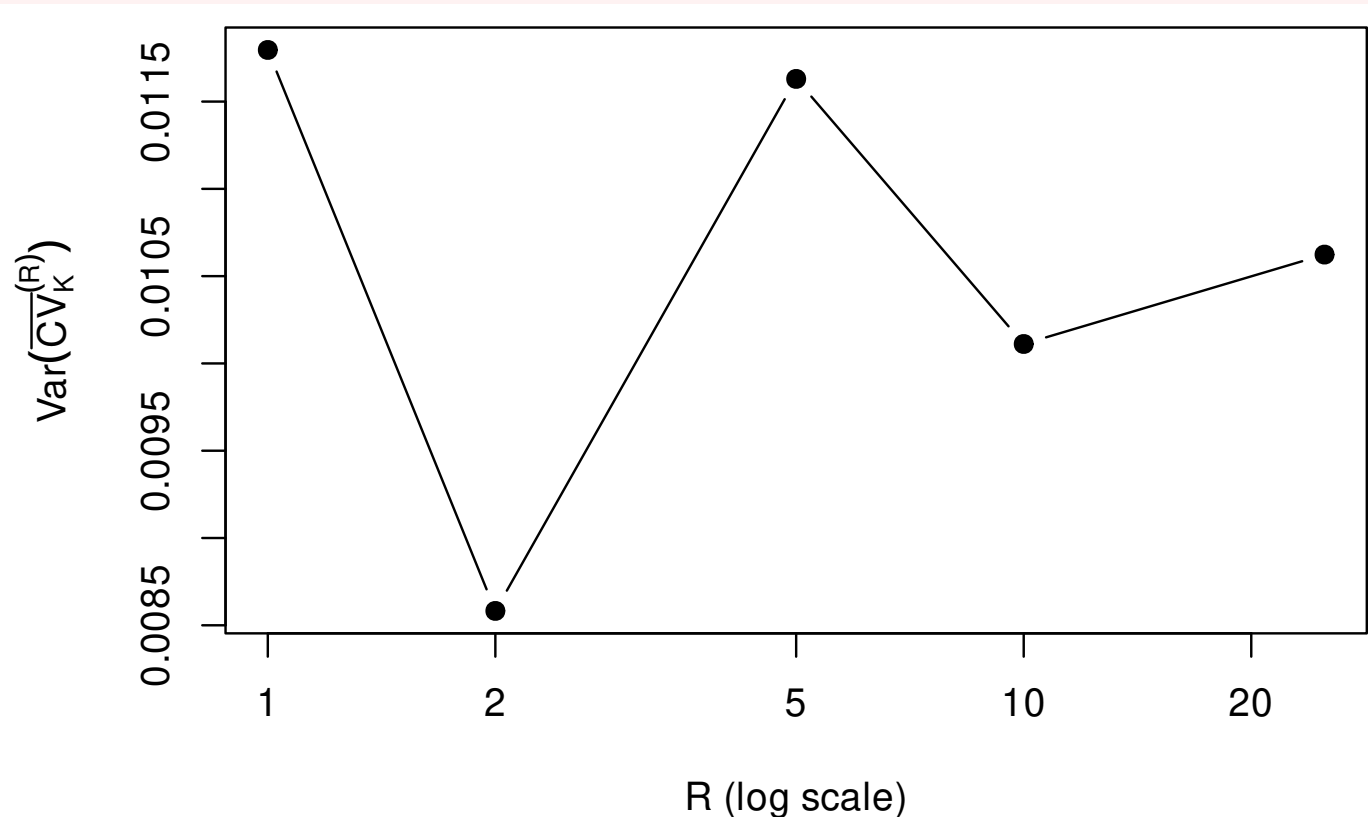
Example 4.5 (Diminishing returns of repetition for OLS). Same setup as Example 4.4; compute $\overline{\text{CV}}_{10}^{(R)}$ for $R \in \{1, 2, 5, 10, 25\}$ and record the across-data-replicate variance.

```

1 set.seed(1)
2 n <- 200; p <- 5; B <- 100; K <- 10
3 beta <- c(1, -1, 0.5, 0, 0)
4 Rs <- c(1, 2, 5, 10, 25)
5
6 cv_K <- function(X, y) {
```



```
7 fold_id <- sample(rep(1:K, length.out = n))
8 errs <- vapply(1:K, function(k) {
9   tr <- which(fold_id != k); te <- which(fold_id == k)
10  fit <- lm.fit(X[tr, , drop = FALSE], y[tr])
11  mean((y[te] - X[te, , drop = FALSE] %*% fit$coefficients)^2)
12 }, numeric(1))
13 mean(errs)
14 }
15
16 result <- sapply(Rs, function(R) {
17   replicate(B, {
18     X <- matrix(rnorm(n * p), n, p)
19     y <- as.numeric(X %*% beta + rnorm(n))
20     mean(replicate(R, cv_K(X, y)))
21   })
22 })
23
24 par(mar = c(4, 5, 0.5, 0.5))
25 plot(Rs, apply(result, 2, var), type = "b", log = "x", pch = 19,
26       xlab = "R (log scale)", ylab = expression(Var(bar(CV)[K]^{(R)})))
```



For OLS at $n \gg p$, training sets across partitions overlap by $(K - 2)/K = 80\%$ of rows, so the

partition-level variance V_{part} is small relative to V_{data} . The variance of $\overline{\text{CV}}_K^{(R)}$ plateaus quickly.

Working choice: $R \in [5, 25]$ repetitions of $K \in \{5, 10\}$, at cost RK fits. Tidymodels exposes this via `rsample::vfold_cv(v = K, repeats = R)`. For unstable models (single trees, nearest neighbors), V_{part} is larger and repetition helps visibly.

3.5 Stratified K-fold

In classification (Unit 3), per-fold loss depends on the fold's class composition. Random partitioning produces folds whose positive-class proportion fluctuates around $\pi := \mathbf{P}(Y = 1)$, adding variance to CV_K . **Stratified K-fold** fixes the class composition.

Definition 4.14 (Stratified K-fold partition). A K-fold partition $\mathcal{I}_1, \dots, \mathcal{I}_K$ of $\{1, \dots, n\}$ is **stratified by** Y if each fold contains, up to rounding, the same proportion of each class:

$$\frac{|\{i \in \mathcal{I}_k : Y_i = c\}|}{|\mathcal{I}_k|} \approx \frac{|\{i : Y_i = c\}|}{n} \quad \text{for every fold } k \text{ and class } c.$$

For binary Y with class rate π , each fold contains $\approx \pi |\mathcal{I}_k|$ positives.

Under random partitioning, the count of positives in a fold of size $m = n/K$ is $\text{Binomial}(m, \pi)$ with variance $m\pi(1 - \pi)$; under stratification, this count is fixed (up to rounding). For severely imbalanced data, random partitioning can leave a fold with zero positives, making per-fold metrics like precision and F1 undefined.

Example 4.6 (Imbalanced classification: stratification matters). Set $\pi = 0.05$, $n = 400$, $K = 10$. Each fold has $m = 40$ rows; expected positives per fold $m\pi = 2$, with SD $\sqrt{m\pi(1 - \pi)} \approx 1.38$.

```
1 set.seed(1)
2 n <- 400; pi_rate <- 0.05; K <- 10
3 y <- rbinom(n, 1, pi_rate)
4 sum(y)
```

```
## [1] 20
```

```
1 # Random partition: positives per fold
2 fold_random <- sample(rep(1:K, length.out = n))
3 table(fold_random, y)[, 2]
```

```
##  1  2  3  4  5  6  7  8  9 10
##  1  0  1  3  3  3  3  2  1  3
```

```
1 # Stratified partition: shuffle within each class
2 fold_strat <- integer(n)
3 for (cls in unique(y)) {
4   idx <- which(y == cls)
5   fold_strat[idx] <- sample(rep(1:K, length.out = length(idx)))
6 }
7 table(fold_strat, y)[, 2]
```

```
##  1  2  3  4  5  6  7  8  9 10
##  2  2  2  2  2  2  2  2  2  2
```

Random partition produces fold counts spanning the binomial range (0 to 4 here); stratification places ≈ 2 positives in every fold. Folds with zero positives leave precision and F1 undefined.

For balanced regression, stratification has no benefit. For imbalanced classification (Unit 3 §3.5), it is the default: `rsample::vfold_cv(v = K, strata = y)`. Pitfalls: stratifying on a column that leaks the target (cf. Unit 1 §1.7); stratifying on continuous Y without discretizing (`rsample` uses `breaks =`, default 4 bins). For long-tailed Y , four bins put most rows in the middle and leave the tails unbalanced; increase `breaks` or log-transform Y first.

4 Splits That Honor Data Structure

K-fold CV's unbiasedness argument (§3.1) requires the test fold to be independent of the training data. iid sampling makes this automatic. Two structural dependencies need modified split schemes: temporal order (rows arrive in time) and group structure (rows share a unit). A separate issue arises when CV is used both to tune hyperparameters and to report risk; nested CV addresses that.

4.1 Time-series cross-validation

For time-indexed data $(X_1, Y_1), \dots, (X_n, Y_n)$, random K-fold places future rows in training and past rows in testing for some k , so the model uses post-prediction-time information: target leakage in time (Unit 1 §1.7). Time-respecting CV trains on past, tests on future.

Definition 4.15 (Rolling-origin CV). For initial window size n_0 and forecast horizon h , the k -th fold is

$$T_k = \{k, k+1, \dots, n_0 + k - 1\}, \quad V_k = \{n_0 + k, \dots, n_0 + k + h - 1\}, \quad k = 1, \dots, K,$$

with $K = n - n_0 - h + 1$. The training window has fixed size n_0 , shifting forward by one row per fold; the test window of length h lies immediately after. The CV estimator averages the per-fold

losses on V_k .

Definition 4.16 (Growing-window CV). As in Definition 4.15, but with $T_k = \{1, \dots, n_0 + k - 1\}$: the training set accumulates all rows before V_k rather than sliding.

The choice depends on stationarity:

- **Stationary** (joint distribution constant in time): growing-window dominates because more data lowers variance.
- **Non-stationary** (concept drift, regime change): rolling-origin avoids training on outdated rows.

`rsample::rolling_origin(initial =  $n_0$ , assess =  $h$ , cumulative = TRUE/FALSE)` implements both.

4.2 Group K-fold cross-validation

When rows share a structural unit (patient with repeated measurements, customer with multiple transactions, store with daily records), iid fails: rows from the same unit are dependent. Random K-fold splits the same unit's rows across train and test folds, breaking the independence Lemma 4.3

requires; CV_K underestimates risk on a new unit.

Definition 4.17 (Group K-fold partition). Let the n rows be labeled by group $g_1, \dots, g_n \in \{1, \dots, G\}$. A **group K-fold partition** is a partition of the **groups** $\{1, \dots, G\}$ into disjoint subsets $\mathcal{G}_1, \dots, \mathcal{G}_K$; the row partition is $\mathcal{I}_k := \{i : g_i \in \mathcal{G}_k\}$.

Each fold contains all rows from its assigned groups and none from other groups. The test fold's groups are unseen during training, restoring the independence Lemma 4.3 requires.

The estimand is the expected risk on a fresh **unit** drawn from the group-level population, not a fresh row from a known unit. The two differ when within-unit correlation is high. `rsample::group_vfold_cv` (`g = g, v = K`) implements this.

4.3 Nested cross-validation

Cross-validation is used both to estimate risk and to tune hyperparameters λ (ridge λ in Unit 2 §2.5.1, classification threshold in Unit 3 §3.4.2). Reporting $\min_{\lambda} CV_K(\lambda)$ as the risk estimate reuses the test folds for selection: $\hat{\lambda}$ is biased toward folds on which it happens to score well, and the reported number is optimistic.

Definition 4.18 (Nested K-fold CV). Outer K-fold partition $\{\mathcal{I}_k^{\text{out}}\}_{k=1}^{K_{\text{out}}}$. For each k :

1. Let the inner training set be the rows outside $\mathcal{I}_k^{\text{out}}$.
2. Run K_{in} -fold CV on the inner training set across a grid Λ ; pick $\hat{\lambda}_k = \arg \min_{\lambda} \text{CV}_{K_{\text{in}}}(\lambda)$.
3. Refit on the inner training set with $\hat{\lambda}_k$; evaluate on $\mathcal{I}_k^{\text{out}}$.

The nested CV estimate is the average of the outer-fold losses.

Cost: $K_{\text{out}} \cdot K_{\text{in}} \cdot |\Lambda|$ full fits. The common compromise is a single outer train/test split with CV-tuning inside training: cost drops to $K_{\text{in}} \cdot |\Lambda| + 1$ at the cost of a noisier risk estimate (§1.3).

When the inner-outer distinction matters

Nested CV is needed when reporting a final risk number for a tuned procedure, especially in publications and competitions. For exploratory comparison at fixed hyperparameters, plain K-fold suffices.

5 Implementation Tour

5.1 rsample: split objects

`rsample` provides four split functions corresponding to the schemes of §§3–4.

```
1 library(tidymodels)
2 set.seed(2026)
3 n <- 200
4 df <- tibble(
5   x1 = rnorm(n), x2 = rnorm(n), x3 = rnorm(n),
6   store = sample(letters[1:20], n, replace = TRUE),
7   date = as.Date("2025-01-01") + 1:n,
8   y = x1 - 0.5 * x2 + 0.3 * x3 + rnorm(n, sd = 0.5)
9 )
```

Each split function returns a tibble whose `splits` column lists `rsplit` objects; `analysis()` extracts a fold's training rows, `assessment()` its test rows.

```
1 vfold_cv(df, v = 5) # plain K-fold
```

```
## # 5-fold cross-validation
## # A tibble: 5 x 2
##   splits          id
```

```
##      <list>          <chr>
## 1 <split [160/40]> Fold1
## 2 <split [160/40]> Fold2
## 3 <split [160/40]> Fold3
## 4 <split [160/40]> Fold4
## 5 <split [160/40]> Fold5
```

```
1 group_vfold_cv(df, group = store, v = 5)           # group K-fold
```

```
## # Group 5-fold cross-validation
## # A tibble: 5 x 2
##   splits      id
##   <list>      <chr>
## 1 <split [165/35]> Resample1
## 2 <split [153/47]> Resample2
## 3 <split [160/40]> Resample3
## 4 <split [159/41]> Resample4
## 5 <split [163/37]> Resample5
```

```
1 rolling_origin(df, initial = 150, assess = 10,           # time-series CV
2               cumulative = FALSE)
```

```
## # Rolling origin forecast resampling
## # A tibble: 41 x 2
##   splits      id
##   <list>    <chr>
## 1 <split [150/10]> Slice01
## 2 <split [150/10]> Slice02
## 3 <split [150/10]> Slice03
## 4 <split [150/10]> Slice04
## 5 <split [150/10]> Slice05
## 6 <split [150/10]> Slice06
## 7 <split [150/10]> Slice07
## 8 <split [150/10]> Slice08
## 9 <split [150/10]> Slice09
## 10 <split [150/10]> Slice10
## # i 31 more rows
```

Modern rsample sliding APIs

`rsample` $\geq$ 1.2 adds `sliding_period()`, `sliding_index()`, and `sliding_window()` for time-respecting splits. These accept a date or index column and handle irregular timestamps

(gaps, weekends, monthly aggregates) more cleanly than index-based `rolling_origin()`. The latter is retained here for pedagogical clarity (the indices match Definition 4.15 directly); the sliding family is the preferred choice in production pipelines with calendar-aware data.

These share a row-tibble structure (one row per fold); `fit_resamples` and `tune_grid` accept any of them interchangeably. `bootstraps()` provides resamples of the same shape, used in Unit 5 for bagging and Random Forest OOB.

5.2 workflow and preprocessing-in-fold

Standardization, imputation, and feature selection must be computed on the training fold alone. Computing them on $\mathcal{D}$ before splitting leaks each test fold into its own training, biasing CV_K downward. The **recipe** + **workflow** idiom enforces this: a **recipe** stores preprocessing steps; fitting a workflow on a fold's training rows trains the recipe on those rows alone, and the test fold is processed with the train-fitted parameters (means, SDs, imputation values).

```
1 rec <- recipe(y ~ x1 + x2 + x3, data = df) |>
2   step_normalize(all_numeric_predictors())
3
```

```
4 mod <- linear_reg() |> set_engine("lm")
5
6 wf <- workflow() |>
7   add_recipe(rec) |>
8   add_model(mod)
```

`fit_resamples()` runs the workflow on each fold and aggregates per-fold metrics:

```
1 fit_resamples(wf, resamples = vfold_cv(df, v = 5)) |>
2   collect_metrics()
```

```
## # A tibble: 2 x 6
##   .metric .estimator  mean      n std_err .config
##   <chr>   <chr>      <dbl> <int>   <dbl> <chr>
## 1 rmse    standard    0.504     5  0.0287 pre0_mod0_post0
## 2 rsq     standard    0.821     5  0.0147 pre0_mod0_post0
```

The reported metrics (**rmse**, **rsq**) are averages of per-fold values; **std_err** is the across-fold standard deviation divided by $\sqrt{K}$ (heuristic; cf. Theorem 4.11).

5.3 `tune::tune_grid` and `tune::last_fit`

Hyperparameters are declared with `tune()` placeholders; `tune_grid` sweeps a grid across the resamples.

```
1 mod_ridge <- linear_reg(penalty = tune(), mixture = 0) |>
2   set_engine("glmnet")
3
4 wf_ridge <- workflow() |>
5   add_recipe(rec) |>
6   add_model(mod_ridge)
7
8 grid <- grid_regular(penalty(range = c(-3, 1)), levels = 10)
9 res <- tune_grid(wf_ridge, resamples = vfold_cv(df, v = 5), grid = grid)
10
11 show_best(res, metric = "rmse", n = 3)
```

```
## # A tibble: 3 x 7
##   penalty .metric .estimator  mean     n std_err .config
##   <dbl> <chr>    <chr>    <dbl> <int>  <dbl> <chr>
## 1 0.001   rmse      standard  0.502     5  0.0487 pre0_mod01_post0
## 2 0.00278 rmse      standard  0.502     5  0.0487 pre0_mod02_post0
```

```
## 3 0.00774 rmse      standard    0.502      5 0.0487 pre0_mod03_post0
```

The end-to-end protocol uses an outer train/test split, tunes on the training portion, and evaluates once on the test portion (§4.3):

```
1 split <- initial_split(df, prop = 0.8)
2 res_train <- tune_grid(
3   wf_ridge,
4   resamples = vfold_cv(training(split), v = 5),
5   grid      = grid
6 )
7 final_wf <- finalize_workflow(wf_ridge, select_best(res_train, metric = "rmse"))
8 final    <- last_fit(final_wf, split = split)
9
10 collect_metrics(final)
```

```
## # A tibble: 2 x 4
##   .metric .estimator .estimate .config
##   <chr>   <chr>      <dbl> <chr>
## 1 rmse    standard      0.528 pre0_mod0_post0
## 2 rsq     standard      0.754 pre0_mod0_post0
```


`last_fit` fits the finalized workflow on `training(split)` and evaluates on `testing(split)`; the fitted model is accessible via `extract_workflow(final)`.

6 Pitfalls and Misapplications

Each entry names a misuse, demonstrates it on a small example, and gives the diagnostic and correct alternative. These also serve as the trap-question bank for the final project.

6.1 Random CV on time-series data

The misuse: applying `vfold_cv` to time-indexed data. Adjacent rows in time are dependent, and random splits put each test row's neighbors into training; the CV estimate is optimistically biased.

```
1 set.seed(1)
2 n <- 300; t <- 1:n
3 y <- numeric(n); y[1] <- rnorm(1)
4 for (i in 2:n) y[i] <- 0.95 * y[i-1] + rnorm(1, sd = 0.3)
5 df_ts <- tibble(t = t, y = y)
6
7 fold <- sample(rep(1:5, length.out = n))
8 mse_random <- mean(sapply(1:5, function(k) {
9   fit <- lm(y ~ poly(t, 3), data = df_ts[fold != k, ])
```

```
10   mean((df_ts$y[fold == k] - predict(fit, df_ts[fold == k, ]))^2)
11   }))
12
13 mse_rolling <- mean(sapply(0:4, function(k) {
14   fit <- lm(y ~ poly(t, 3), data = df_ts[1:(180 + k * 20), ])
15   te  <- (180 + k * 20 + 1):(200 + k * 20)
16   mean((df_ts$y[te] - predict(fit, df_ts[te, ]))^2)
17   }))
18
19 c(random = mse_random, rolling = mse_rolling)
```

```
##      random    rolling
## 0.5711263 2.5896204
```

Diagnostic. Any column representing time (timestamp, sequence index, transaction date).

Fix. `rolling_origin()` (§4.1).

6.2 Ignoring group structure

The misuse: random K-fold on data where rows share an entity (patient, customer, store). The model memorizes the entity from training rows and predicts well on the same entity's rows in the test fold.

```
1 set.seed(1)
2 n <- 200; G <- 20; groups <- letters[1:G]
3 g_eff <- setNames(rnorm(G, sd = 1), groups)
4 df_g <- tibble(group = sample(groups, n, replace = TRUE), x = rnorm(n))
5 df_g$y <- g_eff[df_g$group] + 0.5 * df_g$x + rnorm(n, sd = 0.3)
6
7 # Predictor: group mean from training (memorizes group effects)
8 predict_gm <- function(train, test) {
9   g_mu <- tapply(train$y, train$group, mean)
10  pred <- g_mu[test$group]
11  pred[is.na(pred)] <- mean(train$y) # unseen-group fallback
12  pred
13 }
14
15 fold <- sample(rep(1:5, length.out = n))
16 mse_random <- mean(sapply(1:5, function(k) {
17   mean((df_g$y[fold == k] - predict_gm(df_g[fold != k, ], df_g[fold == k, ]))^2)
18 })))
19
20 g_fold <- split(groups, rep(1:5, length.out = G))
```

```
21 mse_group <- mean(sapply(1:5, function(k) {  
22   in_te <- df_g$group %in% g_fold[[k]]  
23   mean((df_g$y[in_te] - predict_gm(df_g[!in_te, ], df_g[in_te, ]))^2)  
24 })))  
25  
26 c(random = mse_random, group = mse_group)
```

```
##      random      group  
## 0.3758735 1.1094796
```

Diagnostic. Any column with multiple rows per value.

Fix. `group_vfold_cv()` (§4.2).

6.3 Preprocessing outside CV folds

The misuse: computing preprocessing statistics (means, SDs, imputation values, PCA loadings) on $\mathcal{D}$ before splitting. The magnitude varies: a global mean barely differs from per-fold means; target-driven imputation or PCA loadings leak substantially; feature selection by correlation with y (next pitfall) is the dramatic case.

Diagnostic. Any preprocessing step using rows outside the current fold's training set.

Fix. `recipe` + `workflow` (§5.2) computes preprocessing per fold.

6.4 Feature selection outside CV

The misuse: pre-screen features by correlation with y on the whole dataset, then run CV with the selected features. Features were chosen to predict y , so they continue to predict y on every fold by construction.

```
1 set.seed(1)
2 n <- 200; p <- 100
3 X <- matrix(rnorm(n * p), n, p)
4 y <- rnorm(n)                                # no signal at all
5 fold <- sample(rep(1:5, length.out = n))
6
7 # Pre-screen on full data, then CV
8 top5_global <- order(abs(cor(X, y)), decreasing = TRUE)[1:5]
9 mse_global_fs <- mean(sapply(1:5, function(k) {
10   Xtr <- X[fold != k, top5_global]; Xte <- X[fold == k, top5_global]
11   fit <- lm.fit(cbind(1, Xtr), y[fold != k])
12   mean((y[fold == k] - cbind(1, Xte) %*% fit$coefficients)^2)
13 })))
14
```

```

15 # Select within each training fold
16 mse_in_fold_fs <- mean(sapply(1:5, function(k) {
17   top5 <- order(abs(cor(X[fold != k, ], y[fold != k])), decreasing = TRUE)[1:5]
18   Xtr <- X[fold != k, top5]; Xte <- X[fold == k, top5]
19   fit <- lm.fit(cbind(1, Xtr), y[fold != k])
20   mean((y[fold == k] - cbind(1, Xte) %*% fit$coefficients)^2)
21 })))
22
23 c(global_fs = mse_global_fs, in_fold_fs = mse_in_fold_fs)

```

```

## global_fs in_fold_fs
## 0.8301662 0.9930073

```

Pre-screening makes a noise-only target appear predictable.

Diagnostic. Any selection step that uses y .

Fix. Include selection in the recipe (`step_select_*`) so it runs per fold.

6.5 Reporting the best tuned CV score as the test score

The misuse: tune a hyperparameter λ over a grid by CV, then report $\min_{\lambda} CV_K(\lambda)$ as the test error. The chosen $\hat{\lambda}$ is biased toward folds on which it scored low; the minimum is therefore optimistic.

```
1 set.seed(1)
2 B <- 50; n <- 100; p <- 20
3 lambdas <- 10^seq(-3, 1, length.out = 30)
4
5 bias_sim <- replicate(B, {
6   X <- matrix(rnorm(n * p), n, p); y <- rnorm(n)
7   X_te <- matrix(rnorm(1e4 * p), 1e4, p); y_te <- rnorm(1e4)
8   fold <- sample(rep(1:5, length.out = n))
9   cv_curve <- sapply(lambdas, function(lam) {
10     mean(sapply(1:5, function(k) {
11       bhat <- solve(crossprod(X[fold != k, ]) + lam * diag(p),
12                     crossprod(X[fold != k, ], y[fold != k]))
13       mean((y[fold == k] - X[fold == k, ] %*% bhat)^2)
14     }))
15   })
16   bhat_full <- solve(crossprod(X) + lambdas[which.min(cv_curve)] * diag(p),
17                     crossprod(X, y))
18   c(min_cv = min(cv_curve), test = mean((y_te - X_te %*% bhat_full)^2))
19 })
```

```
20 rowMeans(bias_sim)
```

```
##      min_cv      test  
## 1.231806 1.175301
```

Two biases combine: K-fold pessimism inflates CV scores (training on $n(K-1)/K$ rows, §3.2); selection over the grid deflates the minimum. Their net direction varies by setup (here pessimism wins; in higher-dimensional or more flexible settings, selection wins). Either way, the minimum CV is not a fair estimate of the tuned model's risk.

Diagnostic. A tuning loop has selected $\hat{\lambda}$, and the CV score at $\hat{\lambda}$ is being reported.

Fix. Nested CV or a held-out test split (§4.3).

6.6 Across-fold SD treated as the standard error

The misuse: reporting the across-fold SD divided by $\sqrt{K}$ as the standard error of CV_K . The formula assumes independent folds; K-fold folds are correlated through shared training data (§3.3), so it understates the true standard error.

```
1 set.seed(1)  
2 B <- 100; n <- 200; p <- 5  
3 beta <- c(1, -1, 0.5, 0, 0)
```



```

4
5 run_one <- function() {
6   X <- matrix(rnorm(n * p), n, p); y <- as.numeric(X %*% beta + rnorm(n))
7   fold <- sample(rep(1:5, length.out = n))
8   errs <- sapply(1:5, function(k) {
9     bhat <- lm.fit(X[fold != k, ], y[fold != k])$coefficients
10    mean((y[fold == k] - X[fold == k, ] %*% bhat)^2)
11  })
12  c(cv = mean(errs), reported_se = sd(errs) / sqrt(5))
13 }
14 out <- t(replicate(B, run_one()))
15 c(reported_se_mean = mean(out[, "reported_se"]),
16   empirical_se      = sd(out[, "cv"]))

```

```

## reported_se_mean      empirical_se
##           0.09213131          0.11183089

```

The empirical standard deviation of CV_K across data replicates exceeds the reported across-fold SE.

Diagnostic. Any inferential claim (p -value, confidence interval) computed from a CV reported SE.

Fix. Repeat CV across independent data samples (Theorem 4.11).

6.7 SMOTE outside CV folds

The misuse: apply SMOTE (synthetic minority oversampling, Unit 3 §3.5.2) to the full dataset, then run CV. SMOTE creates synthetic minority points by interpolating between real minority neighbors; if the real neighbors include test rows, the synthetic points leak the test target distribution into training.

```
1 library(themis)
2 set.seed(1)
3 n <- 400
4 df_imbal <- tibble(
5   x1 = rnorm(n), x2 = rnorm(n),
6   y = factor(rbinom(n, 1, plogis(-3 + 0.5 * x1))) # ~5% positive
7 )
8 mod <- logistic_reg() |> set_engine("glm")
9
10 # Wrong: SMOTE on full data first, then CV the result
11 df_smoted <- recipe(y ~ ., data = df_imbal) |>
12   step_smote(y) |> prep() |> bake(new_data = NULL)
13 auc_wrong <- collect_metrics(fit_resamples(
14   workflow() |> add_model(mod) |> add_formula(y ~ x1 + x2),
15   resamples = vfold_cv(df_smoted, v = 5),
```

```

16   metrics    = metric_set(roc_auc)
17 ))$mean
18
19 # Right: SMOTE in recipe applied per fold by fit_resamples
20 auc_right <- collect_metrics(fit_resamples(
21   workflow() |>
22     add_recipe(recipe(y ~ ., data = df_imbal) |> step_smote(y)) |>
23     add_model(mod),
24   resamples = vfold_cv(df_imbal, v = 5),
25   metrics    = metric_set(roc_auc)
26 ))$mean
27
28 c(wrong = auc_wrong, right = auc_right)

```

```

##      wrong      right
## 0.6414154 0.5620814

```

Diagnostic. Any class-balancing step (SMOTE, ROSE, undersampling) that uses *y*.

Fix. `themis::step_smote(y)` inside a recipe inside a workflow; resampling then runs per fold (Unit 3 §3.7.5).